Loops

A loop of unknown length becomes a fixed stack of masked layers

---

Why loops are the awkward part

A neural network is a fixed pipeline: the number of layers is chosen before it ever sees an input. A Python loop is the opposite. `for i in range(n)` runs `n` times, `while a != b` runs until something becomes true, and `break` leaves early. The number of iterations depends on the data.

Chapter 1 gave us one trick for control flow: an `if` becomes a 0/1 **mask** plus a **select**. This chapter needs exactly one more idea, and then it is just that trick repeated.

> **Key idea**
>
> **Run the loop body a fixed number of times, and let a 0/1 flag switch off every iteration the real loop would not have done.**
>
> 1. Pick a bound K on the number of iterations. It must hold for *every* input in the declared range: either read it off the range (`n` is at most 5) or prove it (GCD needs at most 11 steps).
> 2. Write the body once as a function from state to state, with no `if` left in it (Chapter 1).
> 3. For each of the K copies, compute a flag $\text{live}_k \in \{0, 1\}$: "would the real loop still be running here?"
> 4. Replace `state = body(state)` with `state = state if not live else body(state)`. That is a big-M select, because the new state is a runtime value and the flag is a runtime 0/1.
> 5. Stack the K copies. The network now has about K times the depth of one body, **whatever the actual iteration count turns out to be**.

The three loop constructs differ only in where the flag live_k comes from:

Python	the flag "live" at copy k	what ends the loop
`for i in range(n)`	$\text{live}_i = \text{step}(n - i)$: a constant compared with the input n	the counter reaches n
`while cond(state)`	$\text{live}_k = \text{cond}(\text{state}_k)$: recomputed from the state each time	the condition turns false; "halted" = live negated
`break` / early `return`	$\text{live}_i = \text{NOT found}_i$: a running OR of the earlier iterations' conditions	a match was seen

The three examples below are these three rows, from the simplest to the most useful.

07 A bounded for loop

basic_python/02_loops/07_bounded_for.py

The problem

Add up the first `n` entries of a list, where `n` itself is an input between 0 and 5:

```
total = 0
for i in range(n):
    total += xs[i]
```

`xs` has 5 entries, each an integer in $[-2, 2]$ (negative on purpose: it makes the big-M select earn its keep). A network cannot run "n times", so it will run 5 times, always, and use the mask $m_i = \text{step}(n - i)$ to keep only the first n additions.

Stage A against stage B

A · control flow

```
def control_flow(n, xs):
    total = 0
    for i in range(n):
        total += xs[i]
    return total
```

B · branchless arithmetic

```
def branchless(n, xs):
    total = 0
    for i in range(N):      # N = 5
        active = step(n - i)
        total = total + select(
            active, xs[i], XB)
    return total
```

The only things that changed are the loop bound (`n` became the constant `N`) and the guarded addition. Both come from the recipe:

- **The flag.** The real loop does iteration i exactly when $i < n$. For integers that is $n - i \geq 1$, which is $\text{step}(n - i)$: it is 1 for $i = 0, \ldots, n-1$ and 0 afterwards. Here the iteration number i is a *constant* (it is a position in the unrolled network), and only n is runtime, so the mask costs two neurons and no multiplication.
- **The guarded add.** "Add `xs[i]` or add nothing" is $\text{total} + \text{select}(m_i, x_i)$, where $\text{select}(m, x) = \text{relu}(x - M(1 - m)) - \text{relu}(-x - M(1 - m))$ is the Chapter 1 big-M select. Only x_i is ever masked, and $|x_i| \leq 2$, so $M = 2$ is enough. (The running total can reach 10, but it is never fed through a select, so it does not influence M.)

A worked trace

Take $n = 3$ and `xs = [2, -1, 1, 2, -2]`. First the masks, then the five unrolled copies of the body:

i	$n - i$	$m_i = \text{step}(n - i)$	x_i	$\text{select}(m_i, x_i)$	total before	total after
0	3	1	2	2	0	2
1	2	1	−1	−1	2	1
2	1	1	1	1	1	2
3	0	0	2	0	2	2
4	−1	0	−2	0	2	2

The grey rows are the iterations the real loop never did. They still run, but they add $\text{select}(0, x) = 0$. Let us check the last one by hand: $x_4 = -2$, $m_4 = 0$, $M = 2$, so $\text{relu}(-2 - 2 \cdot 1) - \text{relu}(2 - 2 \cdot 1) = \text{relu}(-4) - \text{relu}(0) = 0$. For a mask of 0 the arguments of both relus are at most $x - M \leq 0$ and $-x - M \leq 0$ whatever x is (that is exactly what $|x| \leq M$ buys), so both relus are dead.

The masks for every n form a staircase. Row n has ones in columns $0, \ldots, n-1$:

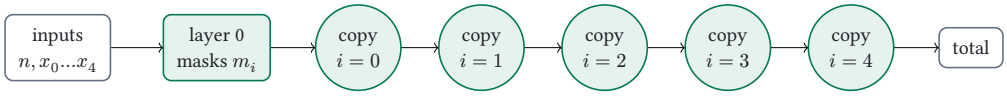
n/i	0	1	2	3	4
0	0	0	0	0	0
1	1	0	0	0	0
2	1	1	0	0	0
3	1	1	1	0	0
4	1	1	1	1	0
5	1	1	1	1	1

Stage C: N copies of one layer

Each state vector between layers is a plain signed vector. A layer is "affine, relu, affine":

$$s' = V \cdot \mathrm{relu}(Ws + b) + c.$$

The network is a chain of $1 + N$ such layers, drawn here for $N = 5$:



The state is $s = [m_0, \ldots, m_4, x_0, \ldots, x_4, \text{total}]$ (11 numbers). Layer 0 builds it from the inputs, and every copy i reads m_i and x_i. The copies are **the same layer with one index changed**: the weight that picks out m_i and x_i moves one column to the right each time.

Layer 0, shown for N = 3 so it fits on the page

Hidden neurons $\mathrm{relu}(n - i)$ and $\mathrm{relu}(n - i - 1)$ give $m_i = \mathrm{step}(n - i)$ by subtracting them in the output map. Three more neurons carry the x_j along (see "pass-through" below).

	n	x0	x1	x2	b
relu(n−0)	1	·	·	·	·
relu(n−0−1)	1	·	·	·	−1
relu(n−1)	1	·	·	·	−1
relu(n−1−1)	1	·	·	·	−2
relu(n−2)	1	·	·	·	−2
relu(n−2−1)	1	·	·	·	−3
relu(x0+2)	·	1	·	·	2
relu(x1+2)	·	·	1	·	2
relu(x2+2)	·	·	·	1	2

The output map V_0, c_0 turns the nine hidden values into the state $[m_0, m_1, m_2, x_0, x_1, x_2, \text{total}]$ (c_0 is the last column):

	h0	h1	h2	h3	h4	h5	h6	h7	h8	c
m0	1	−1	·	·	·	·	·	·	·	·
m1	·	·	1	−1	·	·	·	·	·	·
m2	·	·	·	·	1	−1	·	·	·	·
x0	·	·	·	·	·	·	1	·	·	−2
x1	·	·	·	·	·	·	·	1	·	−2
x2	·	·	·	·	·	·	·	·	1	−2
total	·	·	·	·	·	·	·	·	·	·

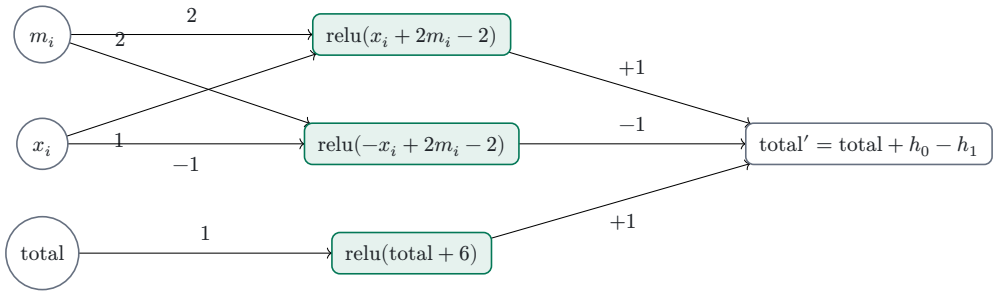
Copy i = 1 (N = 3): one loop iteration as a matrix

Hidden neurons 0 and 1 are the two halves of $\mathrm{select}(m_1, x_1)$ with $M = 2$: their pre-activations are $x_1 + 2m_1 - 2$ and $-x_1 + 2m_1 - 2$. Row 0 reads: weight 2 on m_1, weight 1 on x_1, bias -2. The other seven neurons are pass-throughs.

	m0	m1	m2	x0	x1	x2	total	b
relu(x1−2+2*m1)	·	2	·	·	1	·	·	−2
relu(−x1−2+2*m1)	·	2	·	·	−1	·	·	−2
relu(m0)	1	·	·	·	·	·	·	·
relu(m1)	·	1	·	·	·	·	·	·
relu(m2)	·	·	1	·	·	·	·	·
relu(x0+2)	·	·	·	1	·	·	·	2
relu(x1+2)	·	·	·	·	1	·	·	2
relu(x2+2)	·	·	·	·	·	1	·	2
relu(total+6)	·	·	·	·	·	·	1	6

Output map: the pass-throughs are copied back, and `total` gets $h_0 - h_1 + h_8 - 6$ (the select, plus the old total with its shift undone):

	h0	h1	h2	h3	h4	h5	h6	h7	h8	c
m0	·	·	1	·	·	·	·	·	·	·
m1	·	·	·	1	·	·	·	·	·	·
m2	·	·	·	·	1	·	·	·	·	·
x0	·	·	·	·	·	1	·	·	·	−2
x1	·	·	·	·	·	·	1	·	·	−2
x2	·	·	·	·	·	·	·	1	·	−2
total	1	−1	·	·	·	·	·	·	1	−6



The adding gadget of copy i (weights on the arrows; m_j, x_j for $j \neq i$ are carried by the pass-throughs, not drawn).

Pass-through: keeping a signed number alive through a relu layer

After a relu layer, everything you did not explicitly copy is gone. A value that is always ≥ 0 passes unchanged: $\text{relu}(v) = v$. For a signed value with a known lower bound $-B$, shift it up and shift it back in the next affine map: $\text{relu}(v + B) - B = v$. Here $x_j \geq -2$ and total $\geq -N \cdot 2$, which is why the tables show the biases 2 and 6 (the file has $N = 5$ and uses 10; the tables are drawn for $N = 3$). This costs one neuron per carried value. The same trick will carry all state in every later chapter.

What to notice

- **Depth is N, not n.** The network has $1 + N$ hidden layers for every input, including $n = 0$. If a real run only needs 2 iterations, 3 of the 5 copies are dead weight. That is the price of a fixed shape.
- **The size is easy to count**: layer 0 has 15 neurons, each copy has 13 (2 for the select, $5 + 5 + 1$ pass-throughs): $15 + 5 \times 13 = 80$ neurons.
- **The mask only works because everything is an integer.** $\text{step}(n - i)$ is exactly 0 or 1 only when $n - i$ is an integer; $n = 2.5$ would produce $m = 0.5$ and a half-added element.
- **M must cover the masked value.** If you declared $M = 1$ but an input can be 2, then $\text{select}(0, 2)$ with $M = 1$ gives $\text{relu}(2 - 1) - \text{relu}(-2 - 1) = 1$: the "switched off" iteration leaks into the total. There is no error message; the answer is just wrong. The check `assert` in the file would catch it, which is why the input range is declared (and tested exhaustively) at the top of every file.
- **If the added thing is a constant, no select is needed.** If the body were `total += 2*i + 1`, then masking it is $(2i + 1) \cdot m_i$, a constant times a runtime 0/1 number. That is just a weight. Big-M is only for a mask times a *runtime value* (Exercise 7.1).

Try it

7.1. How would you compute n^2 as "the sum of the first n odd numbers" (`for i in range(n): total += 2*i + 1`)? How many hidden layers does the network need?

7.2. Change the loop to `for i in range(2, n)`: add `xs[i]` for $2 \le i < n$. What changes in stage B?

08 A while loop with a halt flag

`basic_python/02_loops/08_while_halt.py`

The problem

The greatest common divisor by repeated subtraction:

```
while a != b:
    if a > b: a -= b
    else:     b -= a
```

with $a, b \in [1, 12]$. The loop runs between 0 and 11 times depending on the input. Nothing about the inputs tells the network how many layers to use, so we need two things: a safe step count T and a way to make steps after the end harmless.

Choosing T

Every step subtracts the smaller number from the larger one. So the larger value falls: after the step, $\max(a, b)$ is at most the old $\max - 1$ (the new pair is $(\max - \min, \min)$ and $\min \geq 1$). The loop ends at $a = b = \gcd \geq 1$. Starting from at most 12 and ending at least at 1, the maximum drops at most $12 - 1 = 11$ times:

$$T = 11 = \text{HIGH} - 1.$$

It is also tight: $(12, 1)$ takes $12 \to 11 \to \ldots \to 1$, which is 11 steps, and the test file asserts both "no input needs more than T" and "some input needs exactly T". **A T that is too small fails silently** (Exercise 8.2), so the derivation is not optional.

Stage A against stage B

A · control flow

```
def control_flow(a, b):
    iters = 0
    while a != b:
        if a > b:
            a -= b
        else:
            b -= a
        iters += 1
    return a, iters
```

B · branchless arithmetic

```
def branchless(a, b):
    iters = 0
    for _ in range(T):      # T = 11
        halted = eq(a, b)
        gt = step(a - b)
        na = a - select(gt, b)
        nb = b - select(1 - gt, a)
        undo_a = select(halted, a - na)
        undo_b = select(halted, b - nb)
        a, b = na + undo_a, nb + undo_b
        iters = iters + 1 - halted
    return a, iters
```

Reading stage B from the top:

1. **The loop condition becomes a number.** `halted = eq(a, b)` is the *negated* `while` condition as a 0/1 value. It is recomputed from the state at every step, so there is no extra state to carry.
2. **The body runs unconditionally, both branches included** (Chapter 1): `a -= b` is $a - \text{select}(\text{gt}, b)$ and `b -= a` is $b - \text{select}(1 - \text{gt}, a)$. Note that the else-branch also runs when $a = b$, and then it produces $b - a = 0$: a garbage value, because the real loop would never have executed it.
3. **The freeze hides the garbage.** x if m else $y = y + \text{select}(m, x - y)$, with m = halted, x = the old value, y = the proposed new value. When halted is 1 the old state wins and the garbage is discarded; when it is 0, the new state wins.
4. **Counting for free.** Each step adds $1 - \text{halted}$, so `iters` ends as the number of real iterations: the same number stage A counted.

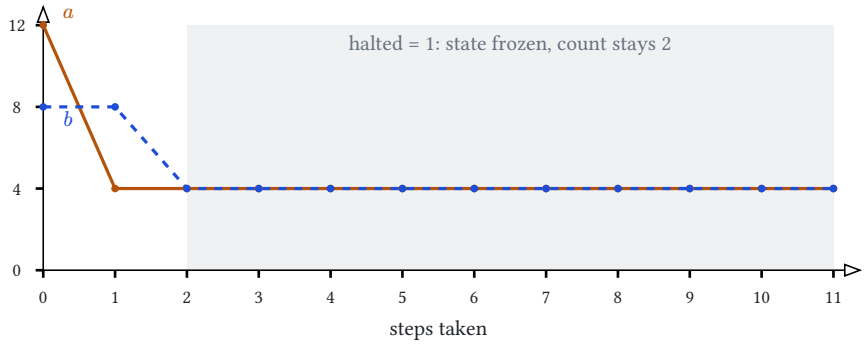
The bound for the freeze select is $M = 12$: it masks $a - a' = \text{select}(\text{gt}, b)$ and $b - b'$, which lie in $[0, 12]$. Every select in the file uses $M = 12$, since all values it touches are within $[-12, 12]$.

A worked trace: gcd(12, 8)

step	a	b	halted	gt	proposed (a', b')	new state	count	
1	12	8	0	1	(4, 8)	(4, 8)	1	real
2	4	8	0	0	(4, 4)	(4, 4)	2	real
3	4	4	1	0	(4, 0)	(4, 4)	2	frozen
4	4	4	1	0	(4, 0)	(4, 4)	2	frozen
5 … 10	4	4	1	0	(4, 0)	(4, 4)	2	frozen
11	4	4	1	0	(4, 0)	(4, 4)	2	frozen

Two real iterations, nine frozen ones, final answer 4 after 11 steps with `count = 2`. The proposed value in the frozen rows is always the garbage $(4, 0)$; the select throws it away every time.

The same thing as a picture: the state of the data-flow run over the 11 steps. The state stops changing after step 2 although the network keeps computing.

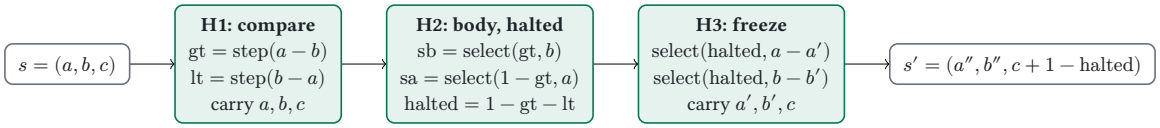


Stage C: one layer, applied T times

The state is $s = [a, b, \text{count}]$. All three entries are non-negative, so a pass-through is a single plain $\text{relu}(v) = v$. One loop step is three hidden relu layers followed by an affine map. **The identical $W_1 \ldots W_4$, $b_1 \ldots b_4$ are used in each of the 11 steps**: the network is a recurrent cell unrolled in time, a weight-tied network:



In file 07 the copies were *similar* (the mask index moved). Here they are literally the same matrices, because the state itself carries everything the step needs: the flag is recomputed from (a, b), never from a position. Inside one step:



Why three hidden layers? Each of the three stages needs the previous stage's output as a *mask* inside a big-M select, and a mask has to be an output of an earlier relu layer before it can be the multiplier input of the next: the comparison must finish before the select, and the select before the freeze. A chain of dependent nonlinear steps costs one layer each.

The weights, exactly as in the file ($M = 12$). Zeros are drawn as dots.

> **H1: compare and carry**
>
> Reads $s = [a, b, \text{count}]$. $\text{gt} = h_0 - h_1$ and $\text{lt} = h_2 - h_3$ are read off in the next matrix.

	a	b	count	b
relu(a−b)	1	−1	·	·
relu(a−b−1)	1	−1	·	−1
relu(b−a)	−1	1	·	·
relu(b−a−1)	−1	1	·	−1
relu(a)	1	·	·	·
relu(b)	·	1	·	·
relu(count)	·	·	1	·

H2: both branches of the body, and the halt flag

Reads h^1. Rows 0 to 3 are the selects: $h_0 - h_1 = \text{select}(\text{gt}, b)$ (because row 0 is $b - 12 + 12\ \text{gt}$ with $\text{gt} = h_0^1 - h_1^1$), and $h_2 - h_3 = \text{select}(1 - \text{gt}, a)$ (row 2 is $a - 12\ \text{gt}$, which is $a - 12(1 - (1 - \text{gt}))$). For non-negative b the second neuron of each pair is always zero (it is there so the same recipe works for signed values). The last row computes the flag $\text{halted} = 1 - \text{gt} - \text{lt}$.

	gt+	gt−	lt+	lt−	a	b	count	b
relu(b−12+12∗gt)	12	−12	·	·	·	1	·	−12
relu(−b−12+12∗gt)	12	−12	·	·	·	−1	·	−12
relu(a−12∗gt)	−12	12	·	·	1	·	·	·
relu(−a−12∗gt)	−12	12	·	·	−1	·	·	·
relu(a)	·	·	·	·	1	·	·	·
relu(b)	·	·	·	·	·	1	·	·
relu(count)	·	·	·	·	·	·	1	·
relu(1−gt−lt)	−1	1	−1	1	·	·	·	1

H3: freeze, and carry the proposed state

Reads h^2. The proposed values are $a' = a - h_0 + h_1$ and $b' = b - h_2 + h_3$. The differences "old minus proposed" are $d_a = a - a' = h_0 - h_1$ and $d_b = h_2 - h_3$, and rows 0 to 3 are $\text{select}(\text{halted}, d_a)$ and $\text{select}(\text{halted}, d_b)$ with $M = 12$ (all four rows have weight 12 on h_7 = halted). Rows 4 to 7 carry a', b', count and halted.

	sb+	sb−	sa+	sa−	a	b	count	halted	b
relu(da−12+12∗h)	1	−1	·	·	·	·	·	12	−12
relu(−da−12+12∗h)	−1	1	·	·	·	·	·	12	−12
relu(db−12+12∗h)	·	·	1	−1	·	·	·	12	−12
relu(−db−12+12∗h)	·	·	−1	1	·	·	·	12	−12
relu(new_a)	−1	1	·	·	1	·	·	·	·
relu(new_b)	·	·	−1	1	·	1	·	·	·
relu(count)	·	·	·	·	·	·	1	·	·
relu(halted)	·	·	·	·	·	·	·	1	·

Output map: the next state

$a'' = a' + \text{select}(\text{halted}, d_a)$ and similarly for b; the count adds $1 - \text{halted}$.

	da+	da−	db+	db−	a'	b'	count	halted	b
a	1	−1	·	·	1	·	·	·	·
b	·	·	1	−1	·	1	·	·	·
count	·	·	·	·	·	·	1	−1	1

Let us push one real step and one frozen step through these matrices.

Step 1, state $(12, 8, 0)$. H1: $h^1 = [4, 3, 0, 0, 12, 8, 0]$, so $\text{gt} = 4 - 3 = 1$ and $\text{lt} = 0 - 0 = 0$. H2: the four select rows have pre-activations $b - 12 + 12\ \text{gt} = 8$, $-b - 12 + 12\ \text{gt} = -8$, $a - 12\ \text{gt} = 0$ and $-a - 12\ \text{gt} = -24$; after the relu that is $[8, 0, 0, 0]$, so $\text{select}(\text{gt}, b) = 8$ and $\text{select}(1 - \text{gt}, a) = 0$. The halt row is $1 - 1 - 0 = 0$, so

halted $= 0$. The proposed state is $a' = 12 - 8 = 4$ and $b' = 8 - 0 = 8$. H3: $d_a = 8$ and the freeze row $d_a - 12 + 12 \cdot \text{halted} = 8 - 12 = -4$ is cut to 0, so $\text{select}(\text{halted}, d_a) = 0$: nothing is undone, and the new state is $(4, 8)$ with count $0 + 1 - 0 = 1$.

Step 3, state $(4, 4, 2)$. H1: both comparisons are 0. H2: halted $= 1 - 0 - 0 = 1$. Row 0 is $b - 12 = -8$, which is cut to 0, so $\text{select}(\text{gt}, b) = 0$ and $a' = 4 - 0 = 4$. Row 2 is $a - 0 = 4$, so $\text{select}(1 - \text{gt}, a) = 4$ and $b' = 4 - 4 = 0$: the garbage. H3: $d_b = 4$, and the freeze row is $d_b - 12 + 12 \cdot 1 = 4$, so $\text{select}(\text{halted}, d_b) = 4$. Output: $b'' = b' + 4 = 4$. The state is back to $(4, 4)$, and the count is $2 + 1 - 1 = 2$.

What to notice

- **The control-flow loop ran 2 times; the network always runs 11.** Depth is $3 \times 11 = 33$ hidden layers, with $7 + 8 + 8 = 23$ neurons each: 253 neurons. This is the price of one fixed shape that works for every input.
- **Weight tying is not an optimisation, it is the design.** The same cell handles step 1 and step 11. You could store the weights once and loop, which is exactly what a recurrent network does; in the file `network` really does `for _ in range(T): s = loop_body(s)`.
- **The halted flag is part of the loop semantics.** Every loop whose body is not harmless after the end condition (here `b -= a` destroys the answer) needs the freeze. In the special case of GCD you could argue that $\text{gt} = \text{lt} = 0$ already stops one branch, but not both (the else-branch still runs); the general recipe does not care, which is why we use it.
- ***T* is a proof obligation.** The code asserts `iterations <= T` over all inputs, and that `T` is attained. If you widen the input range to $[1, 20]$, T becomes 19 and M becomes 20; nothing else changes.
- **The count is a free side effect**: summing $(1 - \text{halted})$ over the steps is itself affine in the flags. Any "how many iterations did it take" or "how many nodes were visited" question is answered this way.

Try it

8.1. What are T and M for $a, b \in [1, 20]$? Which weight matrices change?

8.2. Suppose you wrongly used $T = 5$ for the range $[1, 12]$. What does `network(12, 1)` return as (a, count)?

09 break and early return: the first match

basic_python/02_loops/09_break_first_match.py

The problem

Linear search: the index of the first element equal to `target`, or -1 if there is none.

```
for i in range(5):
    if xs[i] == target:
        return i
return -1
```

`xs` has 5 entries and `target` is an integer, all in $[0, 3]$. Control flow stops at the first match; the data-flow version must look at every element and make sure only the *first* match counts. This is the third loop shape, and the most reused one in the later chapters.

Stage A against stage B

A · control flow

```
def control_flow(xs, target):
    for i in range(N):
        if xs[i] == target:
            return i
    return -1
```

B · branchless arithmetic

```
def branchless(xs, target):
    found = 0
    answer = 0
    for i in range(N):
        match = eq(xs[i], target)
        first = AND(match, 1 - found)
        answer = answer + i * first
        found = found + first
    return answer + found - 1
```

Three numbers per iteration:

- $m_i = (x_i = t) = 1 - \text{step}(x_i - t) - \text{step}(t - x_i)$. Is element i a match?
- found_i: did any *earlier* element match? This is the "break has happened" flag, the `live` flag of the recipe is $1 - \text{found}_i$. It is a running OR.
- $f_i = m_i \text{ AND NOT found}_i = \text{relu}(m_i - \text{found}_i)$ (for 0/1 values $\text{relu}(p + q - 1)$ with $q = 1 - \text{found}$). This is 1 for exactly the first match and 0 for all other positions.

The vector $f = (f_0, \ldots, f_4)$ is a **one-hot vector** (or all zeros): at most one entry is 1, because after the first match $\text{found} = 1$ switches every later f_j off. The running OR is then just a sum, because $f_i = m_i \text{ AND NOT found}$ implies $\text{found OR } m_i = \text{found} + f_i$.

The answer is **affine in f**:

$$\text{index} = \sum_i i \cdot f_i.$$

The multiplication $i \cdot f_i$ is a constant (i is a position in the unrolled network) times a 0/1 number, so there is no big-M. The -1 case: if nothing matched, $f = 0$, so the sum is 0 and $\text{found} = 0$; the final formula is

$$\text{answer} = \sum_i i \cdot f_i + \text{found} - 1.$$

A match at index i gives $i + 1 - 1 = i$; no match gives $0 + 0 - 1 = -1$.

A worked trace

`xs = [2, 0, 3, 0, 3]`, once for `target = 0` (matches at positions 1 and 3) and once for `target = 1` (no match):

i	x_i	m_i target 0	found before	f_i	m_i target 1	found before	f_i
0	2	0	0	0	0	0	0
1	0	1	0	1	0	0	0
2	3	0	1	0	0	0	0
3	0	1	1	0	0	0	0
4	3	0	1	0	0	0	0

For target 0: position 3 is a match too ($m_3 = 1$), but "found before" is already 1, so $f_3 = \text{relu}(1 - 1) = 0$. The answer is $\sum i f_i + \text{found} - 1 = 1 + 1 - 1 = 1$. For target 1: nothing matches, found stays 0, and the answer is $0 + 0 - 1 = -1$.

The "found" flag as a picture, for target 0. It is a staircase that goes up once and stays up; f is the single step of the staircase:

	$i = 0$	1	2	3	4
m_i	0	1	0	1	0
found before i	0	0	1	1	1
f_i (first)	0	1	0	0	0

Stage C

The state is $s = [m_0, ..., m_4, \text{found}, f_0, ..., f_4]$ (11 numbers). Everything in it is 0 or 1, so pass-throughs are plain relus.

Layer M (from the inputs $[x_0, ..., x_4, t]$ to the matches). Four neurons per element give $\text{step}(x_i - t)$ and $\text{step}(t - x_i)$: $\text{relu}(x_i - t)$, $\text{relu}(x_i - t - 1)$, $\text{relu}(t - x_i)$, $\text{relu}(t - x_i - 1)$. The output map computes $m_i = 1 - (h_0 - h_1) - (h_2 - h_3)$ and sets found $= 0$, $f = 0$.

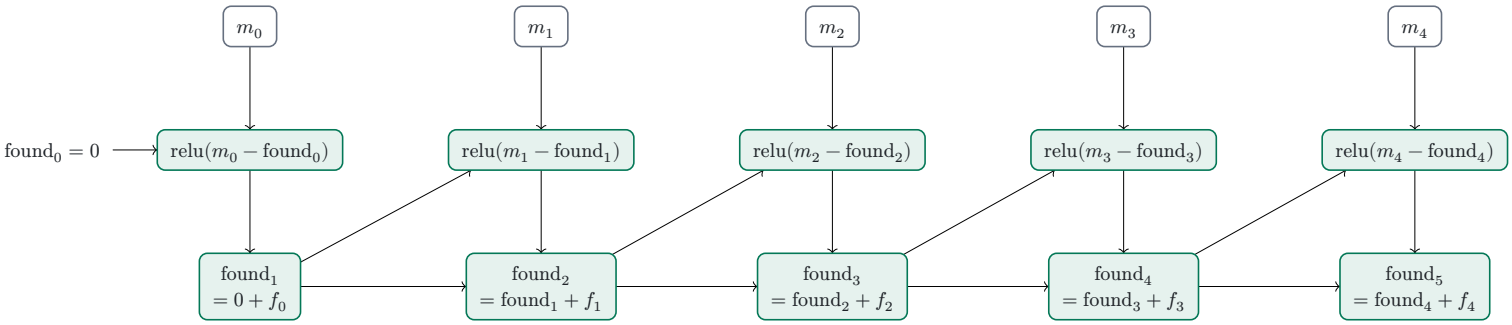
Layer M (N = 3)

	x0	x1	x2	t	b
relu(x0−t)	1	·	·	−1	·
relu(x0−t−1)	1	·	·	−1	−1
relu(t−x0)	−1	·	·	1	·
relu(t−x0−1)	−1	·	·	1	−1
relu(x1−t)	·	1	·	−1	·
relu(x1−t−1)	·	1	·	−1	−1
relu(t−x1)	·	−1	·	1	·
relu(t−x1−1)	·	−1	·	1	−1
relu(x2−t)	·	·	1	−1	·
relu(x2−t−1)	·	·	1	−1	−1
relu(t−x2)	·	·	−1	1	·
relu(t−x2−1)	·	·	−1	1	−1

Output map (c is the last column; the rows for found and f_j are all zero: they start at 0):

	h0	h1	h2	h3	h4	h5	h6	h7	h8	h9	h10	h11	c
m0	−1	1	−1	1	·	·	·	·	·	·	·	·	1
m1	·	·	·	·	−1	1	−1	1	·	·	·	·	1
m2	·	·	·	·	·	·	·	·	−1	1	−1	1	1
found	·	·	·	·	·	·	·	·	·	·	·	·	·
f0	·	·	·	·	·	·	·	·	·	·	·	·	·
f1	·	·	·	·	·	·	·	·	·	·	·	·	·
f2	·	·	·	·	·	·	·	·	·	·	·	·	·

The unrolled loop: one layer per element. Layer i has a single "real" neuron, $f_i = \text{relu}(m_i - \text{found})$, and passes everything else through. Its output map writes f_i into slot i of the one-hot and adds it to found.



Each column is one layer. The "found" flag travels from layer to layer; the f_i computed along the way are collected into the one-hot vector.

Layer i = 1 (N = 3): the loop iteration as matrices

Row 0 is the one non-trivial neuron $\text{relu}(m_1 - \text{found})$ (weight 1 on m_1, weight -1 on found), rows 1 to 7 are pass-throughs. All biases are 0.

	m0	m1	m2	found	f0	f1	f2	b
relu(m1−found)	·	**1**	·	**−1**	·	·	·	·
relu(m0)	**1**	·	·	·	·	·	·	·
relu(m1)	·	**1**	·	·	·	·	·	·
relu(m2)	·	·	**1**	·	·	·	·	·
relu(found)	·	·	·	**1**	·	·	·	·
relu(f0)	·	·	·	·	**1**	·	·	·
relu(f1)	·	·	·	·	·	**1**	·	·
relu(f2)	·	·	·	·	·	·	**1**	·

Output map: copy each pass-through back; `found` gets $h_0 + h_4$ and f_1 gets $h_0 + h_6$. (Only the columns for f_1 and found read h_0; for another i, only the f_i row moves.)

	h0	h1	h2	h3	h4	h5	h6	h7	c
m0	·	**1**	·	·	·	·	·	·	·
m1	·	·	**1**	·	·	·	·	·	·
m2	·	·	·	**1**	·	·	·	·	·
found	**1**	·	·	·	**1**	·	·	·	·
f0	·	·	·	·	·	**1**	·	·	·
f1	**1**	·	·	·	·	·	**1**	·	·
f2	·	·	·	·	·	·	·	**1**	·

Readout. After the last layer, f is one-hot or all-zero. The final affine map is

$$\text{answer} = [0, \ldots, 0, 1 \mid 0, 1, 2, 3, 4] \cdot s - 1,$$

i.e. `found` $+ f \cdot (0, 1, 2, 3, 4) - 1$: a one-hot times `arange(N)`, which is how you decode a one-hot back to an integer in every later chapter.

What to notice

- **The first-one-hot pattern is the reusable part.** "Mark the first element that satisfies something" appears again and again: argmax with ties broken towards the lowest index (Chapter 3), "take the first unvisited node" in a graph search (Chapter 4), "pick the best move" in a game tree (Chapter 5). All of them are $f_i = c_i$ AND NOT any earlier followed by an affine readout $\sum i \cdot f_i$ or $\sum \text{value}_i \cdot f_i$ (the latter needs a big-M select when the values are runtime numbers).
- **`break` is a flag that never resets.** Once found is 1 it stays 1: found only ever gets `+ first`. That is what "the loop has ended" means in data flow.
- **No short-circuit.** Control flow touches only $i + 1$ elements; the network always looks at all 5. The cost is $4N + N(2N + 2)$ neurons (here $20 + 5 \times 12 = 80$), in $N + 1$ hidden layers.
- **Without NOT found, you get a different function:** using m_i instead of f_i in the sum would add up the indices of *all* matches (here $1 + 3 = 4$ for target 0: a plausible-looking wrong answer).

- **The depth does not have to grow with N here.** Chain the layers as above because it mirrors the loop, but see Exercise 9.2 for the shortcut.

Try it

9.1. How do you find the *last* match instead of the first?

9.2. The running OR before position i is $\text{step}(m_0 + \ldots + m_{i-1})$ (a sum of 0/1 numbers is ≥ 1 iff at least one is 1). Use that to compute every $f_i = \text{relu}(m_i - \text{found}_i)$ in parallel. How many hidden layers does the whole network need now, for any list length?

Cost summary

Example	Loop bound	Hidden layers	Neurons	Depends on the actual count?
07 bounded for	$N = 5$ (from the input range)	6	80	no: all 5 copies always run
08 while + halt	$T = 11$ (proved)	33	253	no: 11 steps, frozen after the end
09 break	$N = 5$ (list length)	6	80	no: every element is examined

In every case the depth is set by the **largest** iteration count that can occur, never by the count that does occur. This is the main cost of turning control flow into data flow, and it is also why the design question for your own algorithms starts with "what is the worst case?": the worst case *is* the architecture.

Answers to the exercises

7.1. Each iteration adds the constant $2i + 1$ when it is live: total $= \sum_i (2i+1) \cdot m_i$ with $m_i = \text{step}(n - i) = \text{relu}(n - i) - \text{relu}(n - i - 1)$. The mask times a constant needs no select, so the whole network is **one** hidden layer with $2N$ neurons and an affine readout (the sum is just a weight on each neuron). Check: $n = 3$ gives $1 + 3 + 5 = 9 = n^2$.

7.2. Only the unrolled loop's range changes: iterations $i = 0$ and $i = 1$ are never real, so start the stage-B loop at 2: `for i in range(2, N)`, with the same mask $\text{step}(n - i)$. If $n < 2$ every mask is 0 and the sum is 0, as in Python (`range(2, n)` is empty). The network simply drops the first two copies.

8.1. $T = 19$ (the maximum starts at most at 20 and ends at least at 1) and $M = 20$. In the file only these two constants change. The matrices W_2 and W_3 contain M as an entry, so those two change (their 12s become 20s) and so do the biases that contain $-M$; W_1 and W_4 are untouched.

8.2. `network(12, 1)` with $T = 5$ gives $a = 7$ and `count = 5`: the loop was cut after five subtractions of 1 from 12 and never halted. It is not the gcd (which is 1) and nothing flags the error, which is why the file asserts `iterations <= T`.

9.1. Process the elements from the back: run the unrolled layers in the order $i = 4, 3, 2, 1, 0$. "First" is relative to the order of the layers, so the first match in reverse order is the last match in forward order. Nothing else changes.

9.2. The prefix OR is $\text{found}_i = \text{step}(m_0 + \ldots + m_{i-1})$, one pair of neurons per i (all of them in the same layer, because each depends only on the m's). Then $f_i = \text{relu}(m_i - \text{found}_i)$ is one more layer. Total: three hidden layers (the match bits, the prefix ORs, the f_i), regardless of the length of the list, because the sequential "found" flag was never really sequential. A sequential dependency chain (like GCD) cannot be flattened this way; one that is a pure OR/sum can.